Инструкция пользователя: Фреймворк для моделирования и детектирования гравитационных волн

Разработчик

7 апреля 2026 г.

Аннотация

Данный фреймворк предназначен для симуляции работы мульти-детекторных интерферометров, генерации обучающих выборок, обучения нейронной сети (1D-CNN) для классификации сигналов и визуализации результатов. Система поддерживает как синтетические данные (лазерные ГВ), так и реальные астрофизические симуляции (SXS Catalog).

Содержание

1 Установка и подготовка **2**
1.1 Требования . . . 2
1.2 Структура проекта . . . 2

2 Модуль 1: Генерация датасетов (C++) **2**
2.1 Компиляция . . . 2
2.2 Запуск . . . 2

3 Модуль 2: Конвертер данных SXS (Python) **2**
3.1 Команда запуска . . . 3
3.2 Аргументы . . . 3

4 Модуль 3: Обучение нейросети (Python) **3**
4.1 Команда запуска . . . 3
4.2 Ключевые параметры . . . 3

5 Модуль 4: Визуализация (data_view.py) **3**
5.1 Режим 1: Просмотр из датасета . . . 3
5.2 Режим 2: Просмотр внешнего файла (Новое) . . . 3
5.3 Описание графиков . . . 4

6 Модуль 5: Инференс (predict.py) **4**
6.1 Команды запуска . . . 4
6.2 Вывод результата . . . 4

7 Порядок работы (Workflow) **5**

1 Установка и подготовка

1.1 Требования

- **C++ Compiler:** MinGW (Windows), GCC (Linux) или Clang (macOS). Требуется поддержка стандарта C++17.
- **Python 3.8+:** Установленные библиотеки: `numpy`, `pandas`, `matplotlib`, `torch`, `scikit-learn`, `tqdm`.

1.2 Структура проекта

Рабочая директория должна содержать следующие файлы и папки:

- `multi_detector_generator.cpp` — Генератор датасетов.
- `convert_sxs_to_dataset.py` — Конвертер данных SXS.
- `train_gw_net.py` — Скрипт обучения нейросети.
- `data_view.py` — Визуализатор данных.
- `predict.py` — Скрипт инференса (предсказания).
- `dataset/` — Папка с обучающими данными (создается автоматически).
- `sxs_datasets/` — Папка с экспериментальными данными (опционально).

2 Модуль 1: Генерация датасетов (C++)

Скрипт генерирует временные ряды для двух разнесенных детекторов, имитируя приход гравитационной волны с учетом геометрической задержки и независимого шума.

2.1 Компиляция

Для Windows (Visual Studio или MinGW):

```
g++ -std=c++17 -O2 multi_detector_generator.cpp -o multi_detector.exe
```

2.2 Запуск

```
./multi_detector.exe
```

Результат: В папке `dataset/` создаются подпапки `train/` (80%) и `test/` (20%) с CSV-файлами.

3 Модуль 2: Конвертер данных SXS (Python)

Позволяет загрузить реальные данные слияния черных дыр из каталога SXS и преобразовать их в формат, совместимый с нашей нейросетью.

3.1 Команда запуска

```
python convert_sxs_to_dataset.py --sxs-id SXS:BBH:0305 --output
    bbh_event.csv --noise 1e-22
```

3.2 Аргументы

Аргумент	Описание
`-sxs-id`	ID симуляции (напр. `SXS:BBH:0305`).
`-output`	Имя выходного файла.
`-output-dir`	Папка сохранения (по умолч. `sxs_datasets`).
`-noise`	Уровень шума (RMS) для добавления в сигнал.

4 Модуль 3: Обучение нейросети (Python)

Обучение сверточной сети (1D-CNN) для бинарной классификации (Сигнал/Шум). Используется функция потерь `BCEWithLogitsLoss` для стабильности при малых амплитудах (10^{-22}).

4.1 Команда запуска

```
python train_gw_net.py --epochs 30 --batch-size 64
```

4.2 Ключевые параметры

- `-epochs`: Количество эпох обучения.
- `-batch-size`: Размер мини-батча. Рекомендуется 64 или 32.
- `-device`: `auto`, `cuda` или `cpu`.

Важно: Скрипт автоматически сохраняет лучшую модель в папку `models/best_gw_model.pth`.

5 Модуль 4: Визуализация (data_view.py)

Инструмент для графического анализа данных. Поддерживает два режима работы.

5.1 Режим 1: Просмотр из датасета

```
python data_view.py --id 42 --folder train
```

5.2 Режим 2: Просмотр внешнего файла (Новое)

```
python data_view.py --file sxs_datasets/bbh_event.csv --output plot.png
```

5.3 Описание графиков

Визуализатор строит 5 панелей:

1. **Временной ряд (Дет. 1):** Форма сигнала во времени.
2. **Спектр (Дет. 1):** Частотное представление сигнала.
3. **Временной ряд (Дет. 2):** Сигнал во втором детекторе (со сдвигом).
4. **Спектр (Дет. 2):** Частотное представление второго сигнала.
5. **Кросс-корреляция:** Функция сходства сигналов при различных задержках.

Интерпретация корреляции:

- Зеленая пунктирная линия: *Истинная задержка* (из метаданных).
- *Измеренная задержка* (пик корреляции).
- *Совпадение линий* подтверждает корректность симуляции и наличие сигнала.

6 Модуль 5: Инференс (predict.py)

Скрипт для проверки новых данных с помощью обученной модели.

6.1 Команды запуска

Проверка файла из датасета:

```
python predict.py --id 150 --folder test
```

Проверка произвольного файла:

```
python predict.py --file sxs_datasets/bbh_event.csv
```

Тихий режим (без открытия окна графика):

```
python predict.py --file sxs_datasets/bbh_event.csv --quiet
```

6.2 Вывод результата

Скрипт выводит:

- **Вердикт:** СИГНАЛ или ШУМ (на основе порога 0.5).
- **Уверенность:** Вероятность принадлежности к классу в процентах.
- **Вероятность сигнала:** Сырой выход модели (0.0 - 1.0).

7 Порядок работы (Workflow)

Для воспроизведения полного цикла эксперимента:

1. **Генерация:** Запустите `multi_detector_generator.exe` для создания обучающей выборки.
2. **Обучение:** Запустите `python train_gw_net.py` и дождитесь сохранения модели.
3. **Тест на SXS:** Сконвертируйте данные SXS командой `convert_sxs_to_dataset.py`.
4. **Анализ:** Прогоните полученный файл через `predict.py -file ...` для получения вердикта нейросети.
5. **Визуализация:** Изучите графики через `data_view.py -file ....`